

Week 2 - Day 11: Distributed & Parallel Inference

Tensor Parallelism, Pipeline Parallelism, GPU-Aware Kubernetes & Multi-Node VRAM

Goal: Serve models too large for a single GPU by distributing them across multiple GPUs and nodes. Master tensor parallelism (splitting weight matrices across GPUs within a node), pipeline parallelism (splitting layers across nodes), understand the communication overhead of each, deploy GPU workloads on Kubernetes with the NVIDIA device plugin, and calculate multi-node VRAM budgets correctly.

Prerequisite: Days 9 and 10 (vLLM, PagedAttention, Flash Attention). This day addresses the scaling problem: a 70B model in FP16 needs ~140 GB of VRAM -- more than any single GPU. The techniques here are how every production deployment of frontier-scale models actually works.

PART A — Tensor Parallelism

The Core Idea: Split Matrices Across GPUs

Tensor parallelism (TP) splits individual weight matrices across N GPUs along a chosen dimension. Each GPU holds a shard of every layer and computes its shard of every forward pass in parallel. After each layer a synchronisation step (all-reduce) combines the partial results before the next layer. The key property is that **all GPUs process the same token at the same time** -- every forward pass step involves all GPUs. This means TP adds communication overhead on every step but achieves perfect load balance.

How Weight Matrices Are Split (Megatron-style)

```
# In a transformer MLP block:
#   y = GELU(x @ W1) @ W2   where x: [batch, seq, d_model]
#                               W1: [d_model, d_ffn]
#                               W2: [d_ffn, d_model]
#
# Column-parallel (W1): split W1 along output dimension (columns)
# GPU 0: W1_0 = W1[:, :d_ffn//2] -> partial_y0 = GELU(x @ W1_0)
# GPU 1: W1_1 = W1[:, d_ffn//2:] -> partial_y1 = GELU(x @ W1_1)
# No communication needed after W1 -- each GPU has its half of GELU output
#
# Row-parallel (W2): split W2 along input dimension (rows)
# GPU 0: W2_0 = W2[:d_ffn//2, :] -> partial_z0 = partial_y0 @ W2_0
# GPU 1: W2_1 = W2[d_ffn//2:, :] -> partial_z1 = partial_y1 @ W2_1
# ALL-REDUCE needed: z = partial_z0 + partial_z1 (sum across GPUs)
#
# Communication pattern per MLP block:
#   1 all-reduce of size [batch, seq, d_model] after W2
#   1 all-reduce of size [batch, seq, d_model] after attention output
# Total: 2 all-reduces per transformer layer
#
# For LLaMA-3 8B (32 layers, d_model=4096, batch=1, seq=512, FP16):
#   Per all-reduce volume = 1 x 512 x 4096 x 2 bytes = 4 MB
```

```
# Per layer = 2 x 4 MB = 8 MB
# Full forward pass = 32 x 8 MB = 256 MB per step
# On NVLink (600 GB/s): 256 MB / 600 GB/s = 0.43 ms overhead
# On PCIe (64 GB/s): 256 MB / 64 GB/s = 4.0 ms overhead <- significant!

# Rule: Tensor parallelism ONLY makes sense within a node with NVLink.
# PCIe communication overhead often exceeds the compute savings.
```

Attention Head Splitting in TP

For multi-head attention (MHA), tensor parallelism is applied by assigning whole attention heads to each GPU. Each GPU computes attention for its assigned heads independently (no communication needed within the attention operation itself), then an all-reduce after the output projection combines the results. This is why **num_attention_heads must be divisible by tensor_parallel_size** -- you cannot split half an attention head across GPUs.

Dimension	Formula	Example (LLaMA-3 8B, TP=4)
Heads per GPU	num_heads / TP	32 / 4 = 8 heads/GPU
KV heads per GPU	num_kv_heads / TP	8 / 4 = 2 KV heads/GPU
FFN dim per GPU	d_ffn / TP	14336 / 4 = 3584 units/GPU
Weights per GPU	total_weights / TP	8B / 4 = 2B params/GPU
VRAM per GPU (FP16)	(total_params*2) / TP + buffers	~16GB / 4 + 2GB = 6GB/GPU
All-reduces per layer	2 (MLP + attention)	32 layers x 2 = 64 all-reduces
Communication per step	2 x seq x d_model x bytes x TP	~256 MB per forward pass

Practical Implementation -- Tensor Parallelism with vLLM

A.1 Single-Node Multi-GPU with vLLM

```
# Serve a 70B model across 4 x A100 80GB GPUs (tensor parallelism)
vllm serve meta-llama/Meta-Llama-3-70B-Instruct \
  --tensor-parallel-size 4 \      # split across 4 GPUs (must divide num_heads=64)
  --dtype float16 \
  --gpu-memory-utilization 0.90 \
  --max-model-len 8192 \
  --port 8000

# Verify all 4 GPUs are engaged
nvidia-smi --query-gpu=index,name,memory.used,utilization.gpu --format=csv

# Serve 8B across 2 GPUs (overkill for 8B but useful for demonstrating TP)
vllm serve meta-llama/Meta-Llama-3-8B-Instruct \
  --tensor-parallel-size 2 \
  --dtype float16 \
  --port 8000

# Python API -- offline batch TP
```

```

from vllm import LLM, SamplingParams

llm = LLM(
    model="meta-llama/Meta-Llama-3-70B-Instruct",
    tensor_parallel_size=4,          # must have 4 CUDA devices visible
    dtype="float16",
    gpu_memory_utilization=0.90,
    max_model_len=4096,
)

sp = SamplingParams(max_tokens=200, temperature=0.7)
outputs = llm.generate(["Explain tensor parallelism in detail."], sp)
print(outputs[0].outputs[0].text)

```

A.2 Profile Inter-GPU Communication Bandwidth

```

import torch
import torch.distributed as dist
import time, os

# Measure all-reduce bandwidth between GPUs
# Run with: torchrun --nproc_per_node=4 measure_allreduce.py

def measure_allreduce_bandwidth():
    dist.init_process_group("nccl")
    rank = dist.get_rank()
    world_size = dist.get_world_size()
    device = torch.device(f"cuda:{rank}")
    torch.cuda.set_device(device)

    # Simulate all-reduce for LLaMA-3 8B layer (d_model=4096, seq=512)
    sizes_mb = [1, 4, 16, 64, 256] # MB

    if rank == 0:
        print(f"Measuring all-reduce bandwidth ({world_size} GPUs)")
        print(f"{'Size (MB)':<12} {'Time (ms)':<12} {'BW (GB/s)':<12} {'Effective BW'}")
        print("-" * 52)

    for size_mb in sizes_mb:
        n_floats = size_mb * 1e6 // 2 # FP16 = 2 bytes
        tensor = torch.randn(int(n_floats), dtype=torch.float16, device=device)

        # Warm-up
        for _ in range(5):
            dist.all_reduce(tensor, op=dist.ReduceOp.SUM)
        torch.cuda.synchronize()

        # Timed run
        N_ITERS = 20
        t0 = time.perf_counter()
        for _ in range(N_ITERS):
            dist.all_reduce(tensor, op=dist.ReduceOp.SUM)
        torch.cuda.synchronize()
        elapsed_ms = (time.perf_counter() - t0) * 1000 / N_ITERS

        # Bandwidth: all-reduce moves 2*(N-1)/N * data per GPU (ring all-reduce)

```

```

        bw_gbps = (size_mb * 2 * (world_size-1) / world_size) / (elapsed_ms / 1000) / 1e3

    if rank == 0:
        print(f"{size_mb:<12} {elapsed_ms:<12.3f} {bw_gbps:<12.1f}")

    dist.destroy_process_group()

if __name__ == "__main__":
    measure_allreduce_bandwidth()

# Expected results:
# NVLink (A100 SXM): ~300-600 GB/s effective bandwidth
# PCIe Gen4 x16:      ~30-60 GB/s effective bandwidth
# NVLink is 10x faster -- critical for TP efficiency

```

A.3 Validate TP Scaling Efficiency

```

import time, subprocess, json
from vllm import LLM, SamplingParams

MODEL = "meta-llama/Meta-Llama-3-8B-Instruct"
PROMPTS = ["Explain transformer attention in detail. " * 3] * 32
SP = SamplingParams(max_tokens=150, temperature=0, ignore_eos=False)

results = {}

for tp in [1, 2, 4]: # adjust to available GPUs
    try:
        llm = LLM(model=MODEL, tensor_parallel_size=tp,
                  dtype="float16", gpu_memory_utilization=0.85)
        # Warm-up
        llm.generate(PROMPTS[:4], SP)

        t0 = time.perf_counter()
        out = llm.generate(PROMPTS, SP)
        elapsed = time.perf_counter() - t0

        total_toks = sum(len(o.outputs[0].token_ids) for o in out)
        tps = total_toks / elapsed
        results[tp] = {"tps": tps, "elapsed": elapsed, "toks": total_toks}

        print(f"TP={tp}: {tps:.0f} tok/s ({elapsed:.1f}s {total_toks} tokens)")
        del llm
        import torch; torch.cuda.empty_cache()
    except Exception as e:
        print(f"TP={tp}: skipped -- {e}")

# Scaling efficiency
if 1 in results:
    print("
Scaling efficiency vs TP=1:")
    for tp, r in results.items():
        eff = (r["tps"] / results[1]["tps"]) / tp * 100
        print(f" TP={tp}: {r['tps']:.0f} tok/s efficiency={eff:.0f}%")
    print("Expected: TP=2 -> ~170-190%, TP=4 -> ~300-350% (NVLink node)")
    print(" TP=2 -> ~130-160%, TP=4 -> ~200-270% (PCIe node)")

```

PART B — Pipeline Parallelism

The Core Idea: Split Layers Across Nodes

Pipeline parallelism (PP) divides the model's layers into sequential stages, assigning each stage to a different node (or GPU). Data flows through the stages one after another: node 0 computes layers 1-16, passes the output activation to node 1, which computes layers 17-32, and so on. Unlike tensor parallelism, only one activation tensor is communicated between adjacent stages -- not a full all-reduce across all GPUs. This makes PP suitable for **inter-node** communication (InfiniBand, Ethernet) where bandwidth is much lower than NVLink.

The Pipeline Bubble Problem

The fundamental inefficiency of pipeline parallelism is the **pipeline bubble**: when stage 0 finishes processing a micro-batch and passes it forward, it must wait while all downstream stages process that batch before it can receive the next one. During this wait, stage 0's GPUs are idle. The bubble overhead is:

```
# Pipeline bubble overhead:
#
# With P pipeline stages and M micro-batches:
#   bubble_fraction = (P - 1) / (M + P - 1)
#
# Interpretation:
#   P stages, each forward pass takes 1 unit of time
#   First micro-batch: stages 1..P each wait for the previous stage
#   Last micro-batch: stage P finishes, earlier stages idle waiting for backprop
#   (for inference only the forward pass matters)
#
# Examples:
#   P=2 stages, M=4 micro-batches: bubble = 1/(4+2-1) = 14.3%
#   P=4 stages, M=4 micro-batches: bubble = 3/(4+4-1) = 42.9% <- bad
#   P=4 stages, M=16 micro-batches: bubble = 3/(16+4-1) = 15.8% <- better
#   P=8 stages, M=32 micro-batches: bubble = 7/(32+8-1) = 17.9%
#
# Key insight: bubble overhead decreases as M (batch size) increases.
# For inference (no backprop), continuous batching with large batches
# keeps the bubble small in practice.

def bubble_overhead(n_stages: int, n_microbatches: int) -> float:
    return (n_stages - 1) / (n_microbatches + n_stages - 1)

print("Pipeline bubble overhead:")
print(f"{'Stages':<8} {'Microbatches':<14} {'Bubble %'}")
print("-" * 35)
for stages in [2, 4, 8]:
    for mb in [4, 8, 16, 32]:
        pct = bubble_overhead(stages, mb) * 100
        print(f"{'stages':<8} {'mb':<14} {'pct:.1f}%")
```

TP vs PP: When to Use Which

Dimension	Tensor Parallelism (TP)	Pipeline Parallelism (PP)
-----------	-------------------------	---------------------------


```

        gpu_memory_utilization=0.90,
    )

    sp = SamplingParams(max_tokens=200, temperature=0.7)
    out = llm.generate(["What is pipeline parallelism?"], sp)
    print(out[0].outputs[0].text)

```

B.2 Activation Tensor Communication Between Stages

```

# Between pipeline stages, only ONE activation tensor is communicated:
#   shape: [batch_size, seq_len, hidden_dim]
#   dtype: float16 (or bfloat16)
#
# Communication volume per pipeline boundary crossing:
#   vol_bytes = batch_size x seq_len x hidden_dim x dtype_bytes
#
# For LLaMA-3 70B (hidden_dim=8192), batch=8, seq=512, FP16:
#   vol = 8 x 512 x 8192 x 2 = 67 MB per forward step

def pipeline_comm_volume(batch_size, seq_len, hidden_dim,
                        n_stages, dtype_bytes=2, n_steps=100):
    per_boundary_mb = batch_size * seq_len * hidden_dim * dtype_bytes / 1e6
    n_boundaries    = n_stages - 1
    total_mb        = per_boundary_mb * n_boundaries * n_steps
    return {
        "per_boundary_mb" : per_boundary_mb,
        "boundaries"      : n_boundaries,
        "per_step_mb"     : per_boundary_mb * n_boundaries,
        "total_mb"        : total_mb,
    }

# LLaMA-3 70B: PP=2, TP=4 per node
result = pipeline_comm_volume(8, 512, 8192, n_stages=2)
print(f"Per boundary per step : {result['per_boundary_mb']:.1f} MB")
print(f"Per step total       : {result['per_step_mb']:.1f} MB")

# At InfiniBand 200 Gbps (25 GB/s):
ib_bw_gbs = 25
latency_ms = result["per_boundary_mb"] / 1e3 / ib_bw_gbs * 1000
print(f"InfiniBand latency    : {latency_ms:.1f} ms per step")
print(f"GPU compute (70B step): ~50 ms per step (A100, TP=4)")
print(f"Pipeline overhead      : {latency_ms/50*100:.1f}% of compute time")

```

PART C — GPU-Aware Kubernetes

NVIDIA Device Plugin Architecture

Kubernetes does not natively understand GPUs. The NVIDIA Device Plugin is a DaemonSet that runs on every GPU node, discovers available GPUs via NVML (NVIDIA Management Library), and advertises them to the Kubernetes scheduler as extended resources (nvidia.com/gpu). Once registered, pods can request GPUs using standard resource requests, and Kubernetes guarantees exclusive allocation to the pod -- no GPU sharing between pods unless explicitly configured with MIG (Multi-Instance GPU) or time-slicing.

C.1 Install the NVIDIA Device Plugin

```
# Prerequisites:
# - Kubernetes cluster with GPU nodes
# - NVIDIA drivers installed on each node (nvidia-smi works)
# - nvidia-container-toolkit installed (enables GPU access in containers)

# Install NVIDIA GPU Operator (recommended -- handles driver + device plugin + MIG)
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia
helm repo update

helm install gpu-operator nvidia/gpu-operator \
  --namespace gpu-operator \
  --create-namespace \
  --set driver.enabled=true \           # install/manage drivers
  --set toolkit.enabled=true \         # install container toolkit
  --set devicePlugin.enabled=true \    # install device plugin
  --set mig.strategy=single            # MIG strategy: single, mixed, none

# Or install device plugin only (if drivers/toolkit already present)
kubectl apply -f https://raw.githubusercontent.com/NVIDIA/k8s-device-plugin/v0.14.5/nvidia-device-plugin.yml

# Verify GPUs are registered as allocatable resources on each node
kubectl describe nodes | grep -A 10 "Allocatable"
# Should show: nvidia.com/gpu: 4 (for a 4-GPU node)

# List all GPU nodes
kubectl get nodes -o custom-columns='NAME:.metadata.name,GPU:.status.allocatable.nvidia\.com/gpu'
```

C.2 vLLM Pod Spec for Single-Node Inference

```
# vllm-deployment.yaml -- single node, 4 GPUs, tensor parallel
apiVersion: apps/v1
kind: Deployment
metadata:
  name: vllm-llama3-70b
  namespace: inference
spec:
  replicas: 1
  selector:
    matchLabels:
      app: vllm-llama3-70b
  template:
    metadata:
```



```

labels:
  app: vllm-llama3-70b
spec:
  containers:
  - name: vllm
    image: vllm/vllm-openai:latest
    command: ["python", "-m", "vllm.entrypoints.openai.api_server"]
    args:
      - "--model=meta-llama/Meta-Llama-3-70B-Instruct"
      - "--tensor-parallel-size=4"
      - "--dtype=float16"
      - "--gpu-memory-utilization=0.90"
      - "--max-model-len=8192"
      - "--port=8000"
      - "--enable-prefix-caching"
    env:
      - name: HUGGING_FACE_HUB_TOKEN
        valueFrom:
          secretKeyRef:
            name: hf-token
            key: token
      - name: NCCL_DEBUG
        value: "WARN"
      - name: CUDA_VISIBLE_DEVICES    # explicit GPU assignment
        value: "0,1,2,3"
    resources:
      limits:
        nvidia.com/gpu: "4"          # request 4 GPUs exclusively
        memory: "200Gi"
        cpu: "32"
      requests:
        nvidia.com/gpu: "4"
        memory: "160Gi"
        cpu: "16"
    ports:
      - containerPort: 8000
    volumeMounts:
      - name: model-cache
        mountPath: /root/.cache/huggingface
      - name: shm                    # shared memory for NCCL (required)
        mountPath: /dev/shm
    readinessProbe:
      httpGet:
        path: /health
        port: 8000
      initialDelaySeconds: 120      # model load time
      periodSeconds: 10
    livenessProbe:
      httpGet:
        path: /health
        port: 8000
      initialDelaySeconds: 180
      periodSeconds: 30
      failureThreshold: 3
  volumes:
  - name: model-cache

```

```

    persistentVolumeClaim:
      claimName: model-weights-pvc    # pre-downloaded weights
- name: shm
  emptyDir:
    medium: Memory
    sizeLimit: "32Gi"                # NCCL uses /dev/shm for GPU-GPU comms
nodeSelector:
  accelerator: nvidia-a100           # schedule only on A100 nodes
tolerations:
- key: nvidia.com/gpu
  operator: Exists
  effect: NoSchedule

```

C.3 Service and Horizontal Pod Autoscaler

```

# vllm-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: vllm-service
  namespace: inference
spec:
  selector:
    app: vllm-llama3-70b
  ports:
  - port: 80
    targetPort: 8000
    protocol: TCP
  type: ClusterIP                    # use LoadBalancer for external access

---
# HPA based on custom Prometheus metric (queue depth)
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: vllm-hpa
  namespace: inference
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: vllm-llama3-70b
  minReplicas: 1
  maxReplicas: 4                    # 4 replicas x 4 GPUs = 16 GPUs total
  metrics:
  - type: Pods
    pods:
      metric:
        name: vllm_num_requests_waiting    # from Prometheus adapter
      target:
        type: AverageValue
        averageValue: "10"                # scale up if > 10 waiting requests per pod

```

C.4 Inspect GPU Allocation with kubectl

```

# Check GPU resources allocated to each node

```

```
kubectl describe node <gpu-node-name> | grep -A 20 "Allocated resources"

# List all pods consuming GPUs and their allocation
kubectl get pods -A -o custom-columns=\
'NAMESPACE:.metadata.namespace,NAME:.metadata.name,GPU:.spec.containers[0].resources.limits.nvidia\.com/gpu'

# Describe a vLLM pod to see GPU assignment
kubectl describe pod vllm-llama3-70b-<hash> -n inference

# Watch GPU utilisation from within the pod
kubectl exec -it vllm-llama3-70b-<hash> -n inference -- \
    nvidia-smi --query-gpu=index,name,memory.used,memory.free,utilization.gpu \
    --format=csv --loop=2

# Check vLLM logs for TP/PP initialisation
kubectl logs -f vllm-llama3-70b-<hash> -n inference | grep -E "parallel|block|loaded|worker"

# Port-forward to test locally
kubectl port-forward svc/vllm-service 8000:80 -n inference &
curl http://localhost:8000/health
```

PART D — Multi-Node VRAM Budget Calculation

Complete VRAM Budget Formula

Before provisioning hardware or setting `--max-model-len`, every engineer must calculate the complete VRAM budget. There are four independent consumers of GPU memory in a production vLLM deployment:

```
# VRAM Budget Components (per GPU):
#
# 1. MODEL WEIGHTS
#   weight_vram_per_gpu = total_params x dtype_bytes / TP
#   (pipeline parallelism divides layers; TP divides each layer's weights)
#
# 2. KV CACHE
#   kv_vram = 2 x num_layers_per_stage x num_kv_heads_per_gpu x head_dim
#             x max_seq_len x max_batch_size x kv_dtype_bytes
#   num_layers_per_stage = num_layers / PP
#   num_kv_heads_per_gpu = num_kv_heads / TP
#
# 3. ACTIVATION BUFFERS
#   act_vram = batch_size x seq_len x hidden_dim x dtype_bytes x activation_factor
#   activation_factor ~ 2-4 (intermediate activations during forward pass)
#   Rule of thumb: ~10-15% of weight VRAM at inference
#
# 4. CUDA CONTEXT + NCCL BUFFERS
#   ~0.5-1.0 GB fixed overhead per GPU (CUDA kernels, NCCL comms)

def vram_budget(
    total_params_B,      # model parameter count in billions
    dtype_bytes,         # 2 for FP16, 1 for INT4 (approx)
    tp,                  # tensor parallel degree
    pp,                  # pipeline parallel degree
    num_layers,          # total transformer layers
    num_kv_heads,        # KV attention heads (GQA heads)
    head_dim,            # dimension per head
    max_seq_len,         # context window
    max_batch_size,      # maximum concurrent sequences
    kv_dtype_bytes=2,    # 2=FP16, 1=FP8
    gpu_vram_gb=80,      # target GPU VRAM
):
    layers_per_stage = num_layers / pp
    kv_heads_per_gpu = num_kv_heads / tp

    # 1. Weights
    weight_gb = (total_params_B * 1e9 * dtype_bytes) / tp / 1e9

    # 2. KV Cache
    kv_gb = (2 * layers_per_stage * kv_heads_per_gpu * head_dim *
             max_seq_len * max_batch_size * kv_dtype_bytes) / 1e9

    # 3. Activations (~12% of weights)
    act_gb = weight_gb * 0.12

    # 4. CUDA / NCCL overhead
    overhead_gb = 0.75
```



```
print(f"{ctx_len:<16,} {total:>14,} {concurrent:>20,}")
```

Choosing TP vs PP vs Both: Decision Framework

Scenario	Recommended Config	Reasoning
7B / 8B on 1 GPU (24+ GB)	TP=1, PP=1 (no parallelism)	Single GPU -- no distribution needed
13B on 2 x A100 (NVLink)	TP=2, PP=1	TP within node, NVLink handles comms
70B on 4 x A100 (NVLink, same node)	TP=4, PP=1	Fastest: NVLink all-reduce, no bubble
70B on 2 nodes x 4 GPUs (InfiniBand)	TP=4, PP=2	TP within node, PP across nodes
405B on 4 nodes x 8 GPUs	TP=8, PP=4	Maximise within-node TP, PP for scale
70B, latency-critical (< 500ms TTFT)	TP=8 single node (8xH100 NVLink)	More TP = less per-GPU compute per step
70B, throughput-critical (high QPS)	TP=4, multiple replicas	Scale replicas not TP for throughput

Testing & Metrics

Multi-GPU Validation Checklist

```
# 1. Confirm all GPUs are visible in the pod / machine
nvidia-smi --query-gpu=index,name,uuid --format=csv

# 2. Verify NVLink topology (critical for TP efficiency)
nvidia-smi topo --matrix
# Output shows NVLink (NV1/NV2/NV3) or PCIe (PIX/PXB/PHB) between GPU pairs
# NVLink peers should show NV4+ for best TP performance

# 3. Test NCCL all-reduce bandwidth (run nccl-tests)
git clone https://github.com/NVIDIA/nccl-tests
cd nccl-tests && make MPI=0
./build/all_reduce_perf -b 1M -e 1G -f 2 -g 4 # test on 4 GPUs
# Look for "busbw" column -- should be >300 GB/s on NVLink

# 4. vLLM multi-GPU startup sanity
vllm serve meta-llama/Meta-Llama-3-8B-Instruct --tensor-parallel-size 2 \
  --dtype float16 2>&1 | grep -E "worker|GPU|parallel|loaded"
# Expected lines:
# "Initializing the vLLM engine with config: ... tensor_parallel_size=2"
# "rank 0: using device cuda:0"
# "rank 1: using device cuda:1"
# "Loading model weights..."
# "# GPU blocks: XXXX"

# 5. Load test a TP deployment
python -c "
import asyncio, httpx, time

async def burst(n=50):
    async with httpx.AsyncClient(timeout=120) as c:
        tasks = [c.post('http://localhost:8000/v1/chat/completions',
```

```

        json={'model':'llama3',
              'messages':[{'role':'user','content':'Hello'}],
              'max_tokens':50})
    for _ in range(n)]
    t0 = time.perf_counter()
    rs = await asyncio.gather(*tasks)
    wall = time.perf_counter() - t0
    ok = sum(1 for r in rs if r.status_code == 200)
    toks = sum(r.json()['usage']['completion_tokens']
               for r in rs if r.status_code == 200)
    print(f'{ok}/{n} OK {toks} toks {wall:.1f}s {toks/wall:.0f} tok/s')

asyncio.run(burst(50))
"

# 6. Check Kubernetes GPU allocation is exclusive (no over-subscription)
kubectl describe node <gpu-node> | grep -A 5 "Allocated resources"
# nvidia.com/gpu: X / Y (X allocated, Y total)

```

Key Metrics for Distributed Deployments

Metric	How to Measure	Target
TP scaling efficiency	vllm_tps(TP=N) / (N x vllm_tps(TP=1))	>80% (NVLink), >60% (PCIe)
NCCL all-reduce BW	nccl-tests all_reduce_perf busbw	>300 GB/s (NVLink A100)
Pipeline bubble %	$(P-1) / (M+P-1) \times 100$	<20% with large batch
Inter-node bandwidth	iperf3 between nodes	>12 GB/s (100G IB)
VRAM utilisation per GPU	nvidia-smi memory.used / memory.total	70-90% (headroom for spikes)
GPU idle time	nvidia-smi utilization.gpu	<10% idle under load
Kubernetes GPU allocation	kubectl describe node allocatable	Matches requested GPUs

Key Takeaways

- **Tensor parallelism splits weight matrices within each layer** across N GPUs. All GPUs work every step, communicating via all-reduce. Requires NVLink (300+ GB/s) to be efficient -- PCIe all-reduce overhead can eliminate all benefit. num_attention_heads must be divisible by TP.
- **Pipeline parallelism splits layers into sequential stages** across nodes. Only activation tensors (not all-reduce) cross the boundary -- suitable for slower inter-node networks (InfiniBand / Ethernet). The bubble overhead $(P-1)/(M+P-1)$ decreases with larger micro-batch counts.
- **TP within node, PP across nodes** is the canonical production configuration for models that exceed single-node VRAM (e.g. TP=4, PP=2 for 70B on 2 nodes).
- **The NVIDIA Device Plugin** advertises GPUs as nvidia.com/gpu resources in Kubernetes. Always request exact GPU counts in both requests and limits, mount /dev/shm as tmpfs for NCCL, and set readinessProbe with delay >= model load time.
- **Use the VRAM budget calculator** before every deployment: Weights + KV Cache + Activations + CUDA overhead must fit with 10% headroom.

- **--max-model-len is the primary concurrency lever:** halving max context doubles the number of concurrent sequences the KV cache can support.

Next topic: **Day 12 -- Caching, Cost Optimization & API Layer:** prefix/prompt caching, semantic caching with FAISS/GPTCache, prompt compression with LLMingua, API gateway design with Kong/Traefik, fallback routing, and multi-model routing.